CSE 3201 Software Engineering
Process Models

The waterfall model arranges development as sequential phases: requirements, design, implementation, verification, and maintenance. It is simple to manage but handles changing requirements poorly because earlier phases are hard to revisit.
The spiral model, proposed by Barry Boehm, is driven by risk. Each loop of the spiral plans, analyzes risk, engineers, and evaluates a prototype.
Agile methods favor working software, customer collaboration, and responding to change. Scrum organizes work into fixed-length sprints, usually two to four weeks, with roles of product owner, scrum master, and development team.

CSE 3201 Software Engineering
Requirements Engineering

Requirements engineering establishes what a system should do. Functional requirements describe the behaviour and services of the system. Non-functional requirements describe qualities such as performance, security, and usability.
Requirements are elicited through interviews, surveys, observation, and prototyping, then recorded in a software requirements specification. Validation checks that the documented requirements reflect what stakeholders actually need.
A use case describes a sequence of interactions between an actor and the system that achieves a goal.

CSE 3201 Software Engineering
Software Design Principles

Good modular design aims for high cohesion within modules and low coupling between them. Cohesion measures how closely the responsibilities of a module belong together. Coupling measures how strongly modules depend on each other.
The SOLID principles guide object-oriented design. The single responsibility principle says a class should have only one reason to change. The open-closed principle says software should be open for extension but closed for modification. The Liskov substitution principle says subtypes must be usable in place of their base types. Interface segregation and dependency inversion complete the set.

CSE 3201 Software Engineering
Design Patterns

A design pattern is a reusable solution to a commonly occurring design problem. The Singleton pattern guarantees that a class has only one instance. The Factory Method pattern lets subclasses decide which class to instantiate. The Observer pattern defines a one-to-many dependency so that dependents are notified automatically when an object changes state. The Strategy pattern encapsulates interchangeable algorithms behind a common interface.
The Model-View-Controller architecture separates data, presentation, and input handling.

CSE 3201 Software Engineering
Software Testing

Unit testing verifies individual components in isolation. Integration testing checks how components work together. System testing evaluates the complete product, and acceptance testing confirms that it meets user needs.
Black-box testing derives test cases from the specification without looking at the code, using techniques like equivalence partitioning and boundary value analysis.
White-box testing uses knowledge of the internal structure and measures statement or branch coverage.
Regression testing re-runs existing tests after a change to confirm that nothing has broken. Test-driven development writes a failing test before writing the code.

COCOMO estimates development effort from the size of the software in thousands of lines of code. Function point analysis measures size from the functionality delivered to the user rather than from code length.
Software maintenance is divided into corrective maintenance, which fixes defects; adaptive maintenance, which accommodates a changing environment; perfective maintenance, which adds improvements; and preventive maintenance, which reduces future problems. Refactoring restructures code without changing its external behaviour.